

Security Assessment Report

Cars24 Security Engineering Challenge

Target	https://security-ctf.vercel.app
Engagement Type	Authorized CTF-style web application assessment
Methodology	Manual, targeted testing (black-box, then authenticated)
Report Date	02 October 2026
Report Version	2.0 — Revised: CVSS-scored, PII redacted
Critical Findings	3
Overall Risk Rating	CRITICAL

PREPARED BY

Avinash Kumar Singh

Vellore Institute of Technology, Bhopal

avinash.mar05@gmail.com | avinash.23bcy10006@vitbhopal.ac.in

+91 7498997018

This assessment was performed under the explicit scope and authorization stated on the target application's own login page as part of Cars24's Security Engineering hiring process.

Executive Summary

This report documents the results of a manual, authorized security assessment of the Cars24 Security Engineering Challenge portal, performed in accordance with the scope and rules of engagement published on the application's own login page. Testing was conducted as a logged-in low-privilege user (`cars_user`), using only manual, targeted techniques — no automated scanning, brute-forcing, or denial-of-service activity was performed, consistent with the stated out-of-scope restrictions.

The assessment identified eight notable issues, including three Critical-severity vulnerabilities that, individually or combined, would allow a low-privileged, authenticated attacker to fully compromise the application: bypassing authentication entirely, executing arbitrary operating system commands on the host, and reading other customers' sensitive personal data — including government-issued tax ID (PAN) numbers — with no access control in place.

Business impact. The PAN exposure (Finding 3) is not just a technical IDOR — PAN is a government-issued tax identifier regulated as sensitive personal data under India's Digital Personal Data Protection (DPDP) Act, 2023. An exploitable, unauthenticated-effort IDOR against this field would likely constitute a reportable data breach and regulatory exposure, independent of its raw CVSS technical score — which is why it is rated Critical in this report despite a calculated CVSS base of 6.5 (see Finding 3).

Severity ratings below are calculated per the CVSS v3.1 base metric methodology; where business or regulatory context justifies rating a finding above its raw CVSS band, that is stated explicitly next to the finding. A prioritized remediation roadmap is provided at the end of this report.

Summary of Findings

#	Finding	CVSS Base	Severity	Location
1	SQL Injection leading to Authentication Bypass	9.1	Critical	/dealer-login
2	OS Command Injection	8.8	Critical*	/tools/network
3	IDOR exposing PII (PAN numbers)	6.5	Critical*	/invoices
4	Hardcoded Credentials in Page Source	6.5	Medium	/login
5	Verbose Database Error Disclosure	5.3	Medium	/dealer-login
6	Cleartext Password Input Field	1.8	Low	/dealer-login
7	Admin Access Control Enforced Correctly (Positive)	n/a	Info	/admin
8	Server-Side Template Injection (Tested, Not Exploitable)	n/a	Info	/tools/notification-preview

* CVSS base score falls in the High/Medium band; rated Critical in this report due to exploit-chain impact (Finding 2) or regulatory/PII sensitivity (Finding 3). See each finding for the calibration note.

Scope, Methodology & Tooling

The target application explicitly invites security testing as a CTF-style hiring assessment. **Out of scope:** brute-forcing/credential stuffing, denial-of-service or load/stress testing, and any other disruptive activity. All testing below was manual and targeted, in line with this scope.

Tooling. Testing was performed using browser DevTools (Elements/Network panels, View Source) and manual HTTP request manipulation via the browser address bar and form fields. No intercepting proxy (e.g. Burp Suite) or automated scanner was used for this pass — every payload below was crafted and submitted by hand, consistent with the engagement's no-automated-tooling restriction.

Limitations / Not Tested — the following were explicitly out of scope for this pass and are not covered by this report:

- No source code review — assessment was fully black-box against the running application.
- No testing as `portal_admin` — the account was confirmed to exist via SQLi (Finding 1) but its password was never obtained or guessed, so privilege-escalation impact beyond disclosure is inferred, not demonstrated.
- No automated scanning, fuzzing, or brute-force/credential-stuffing testing (explicitly out of scope).
- No systematic enumeration of the full invoice ID range beyond the single PoC in Finding 3, to avoid unnecessary access to other customers' data.
- Session management, CSRF, and XSS were not a dedicated focus of this testing pass and should be covered in a follow-up assessment.

Access Narrative

Reconnaissance. Initial recon was limited to manually reviewing page source (browser View Source / DevTools) on every page reachable from the public login screen, rather than running an automated crawler or directory brute-forcer — consistent with the engagement's scope restrictions. This manual review of the public, unauthenticated `/login` page source is what surfaced the hardcoded test account (Finding 4) that became the entry point for the rest of this assessment.

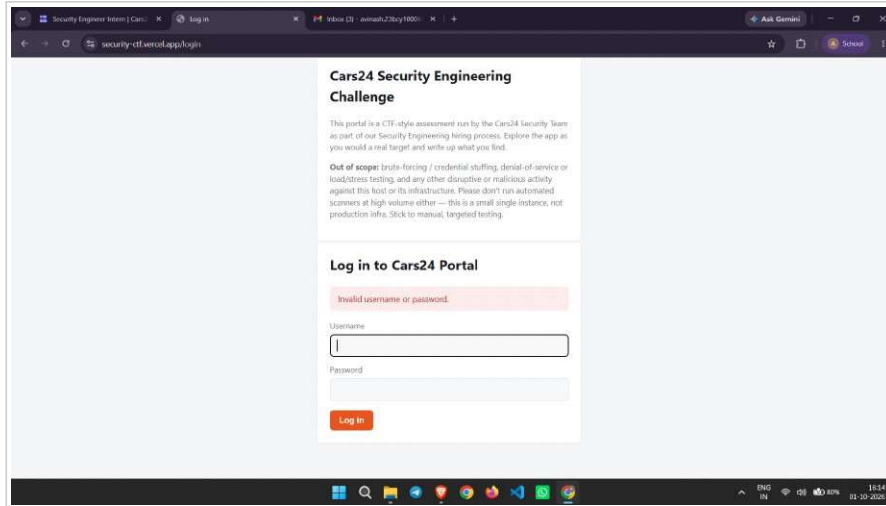


Figure 0.1 — Login page stating the engagement's scope and rules of engagement.

Authenticating with the discovered test account (`cars_user` / `cars_password`) provided access to a dashboard exposing a wide range of internal tools and customer-data features, shown below.

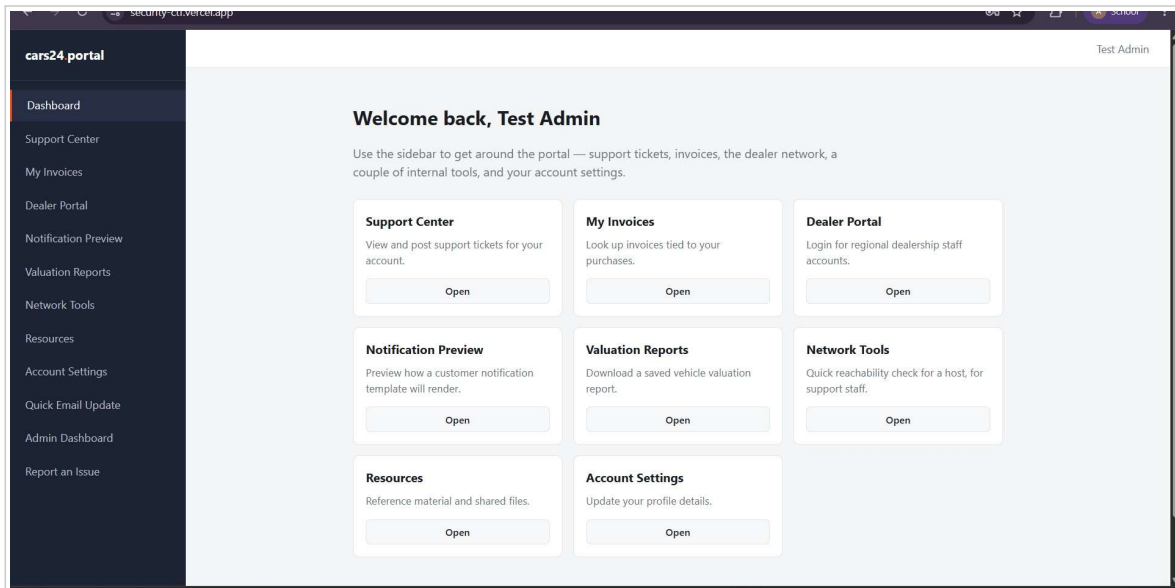


Figure 0.2 — Post-authentication dashboard, listing Support Center, Invoices, Dealer Portal, Notification Preview, Valuation Reports, Network Tools, Resources, Account Settings, and an Admin Dashboard link — a large attack surface for a standard customer-level account.

FINDING 1 — SQL Injection leading to Authentication Bypass

CRITICAL

CWE Reference: CWE-89

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
(9.1)

Location: POST /dealer-login (username / password parameters)

Description

The Dealer Portal login form builds its backend SQL query via direct string concatenation of user-supplied input, with no parameterization or input sanitization. Submitting a classic tautology-based SQL injection payload in the username field bypasses authentication entirely and returns the full contents of the `dealer_users` table — including a privileged administrator account.

Proof of Concept

Username: ' OR '1'='1' --

Password: x (any value)

The application responded with the entire dealer account table in its JSON response, including a row for `portal_admin` with `is_admin: 1`:

```
{
  "rows": [
    {"id": 1, "username": "dealer_mumbai", "dealership": "Cars24 Mumbai Hub",
    "is_admin": 0},
    {"id": 2, "username": "dealer_pune", "dealership": "Cars24 Pune Hub",
    "is_admin": 0},
    {"id": 3, "username": "portal_admin", "dealership": "HQ", "is_admin": 1}
  ],
  "query": "SELECT id, username, dealership, is_admin FROM dealer_users WHERE
username = '' OR
'1'='1' --' AND password = 'x'"
}
```

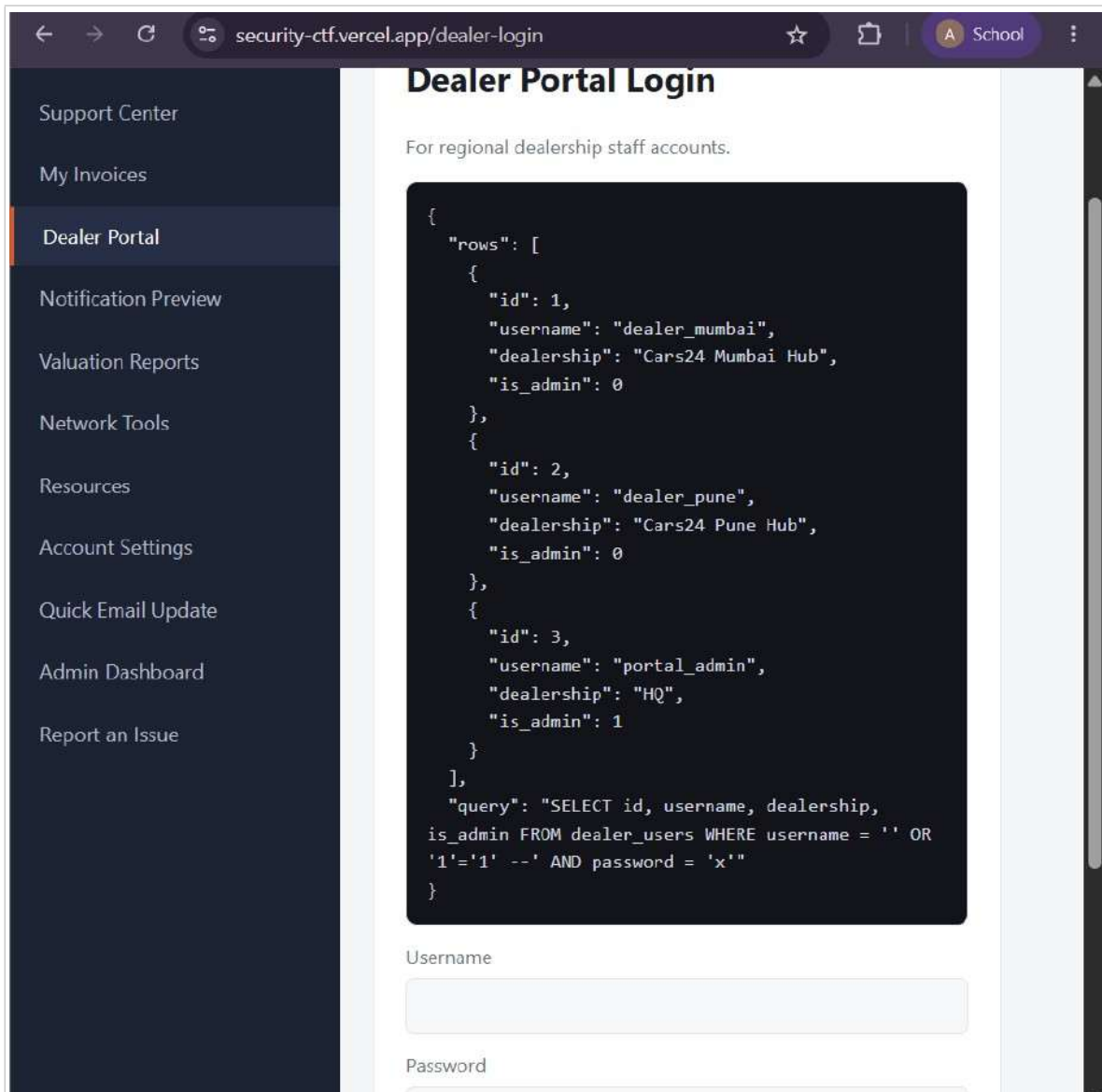


Figure 1.1 — Authentication bypassed via SQL injection; full dealer_users table returned, revealing a privileged portal_admin account (is_admin: 1).

Impact

Complete authentication bypass for the Dealer Portal; disclosure of every dealer account's username, assigned dealership, and admin flag with no valid credentials required. The presence of a confirmed portal_admin account with is_admin=1 indicates an attacker could very likely escalate to full administrative access to the platform.

Remediation

- Use parameterized queries / prepared statements for all database access — never concatenate user input into SQL strings.
- Apply least-privilege database accounts for the web application.

FINDING 2 — OS Command Injection

CRITICAL

CWE Reference: CWE-78

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
(8.8)

Location: POST /tools/network (host parameter)

Description

The Network Tools "host reachability check" feature passes the user-supplied host value directly into a shell command without sanitization. Standard shell metacharacters such as semicolons (;) and && appeared to be filtered or ineffective, but backtick command substitution was not, allowing full arbitrary command execution as the application's runtime user.

Proof of Concept

Payload: 8.8.8.8`echo CTFTEST123`

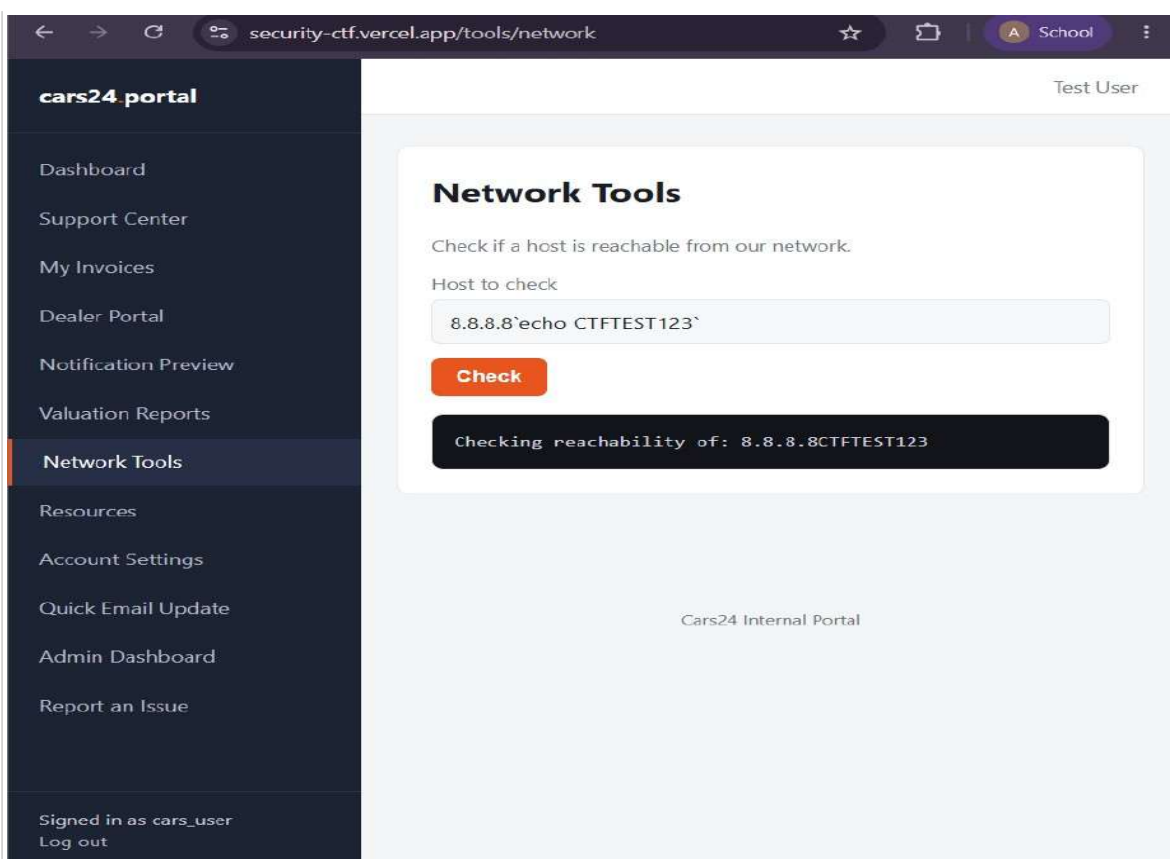


Figure 2.1 — Output shows "CTFTEST123" concatenated onto the IP with the backticks stripped, proving the shell executed the injected command rather than treating it as literal text.

Command execution was further confirmed with standard recon commands:

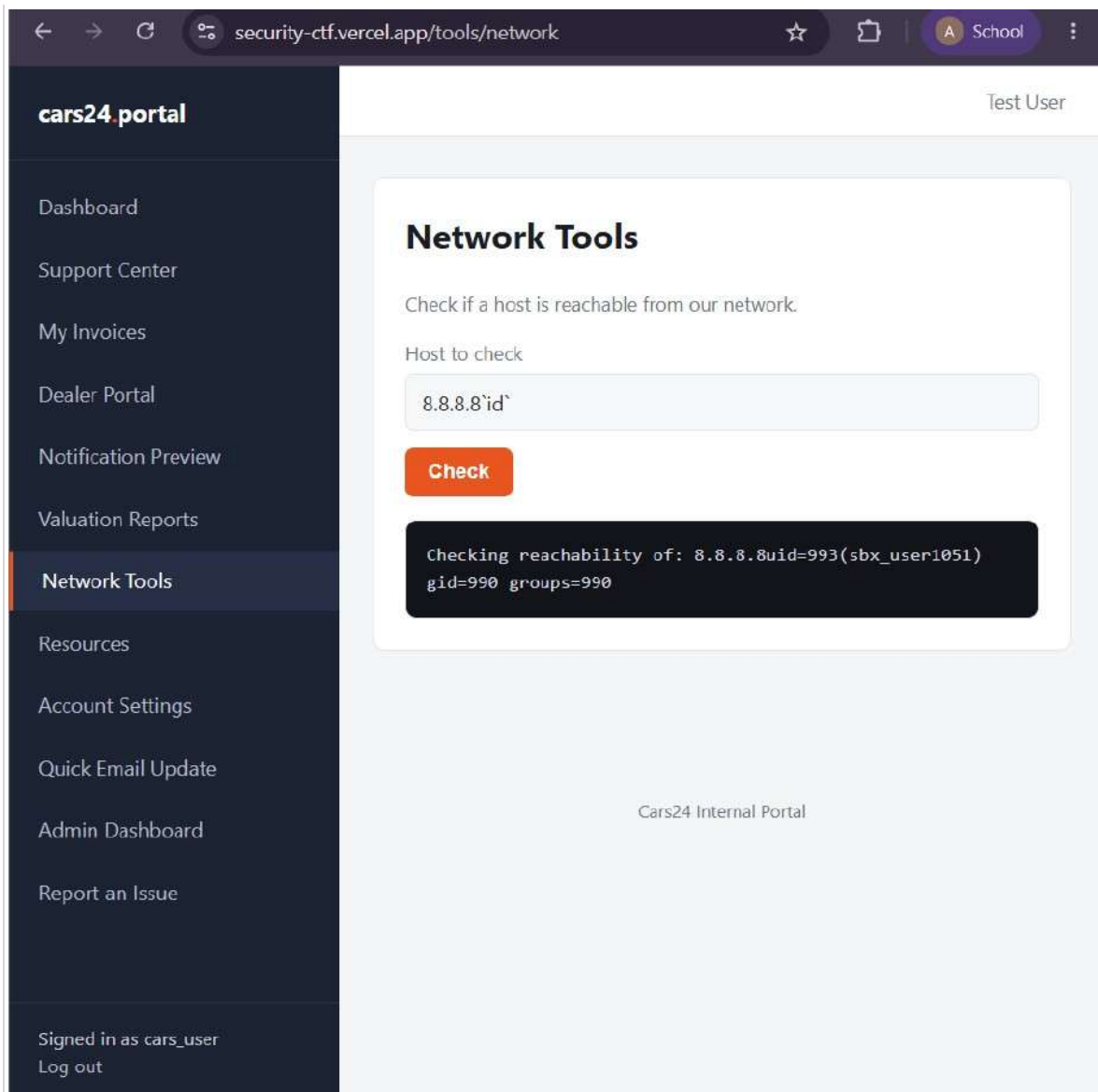


Figure 2.2 — Payload 8.8.8.8`id` returns uid=993(sbx_user1051) gid=990 groups=990, confirming arbitrary command execution with the privileges of the application process.

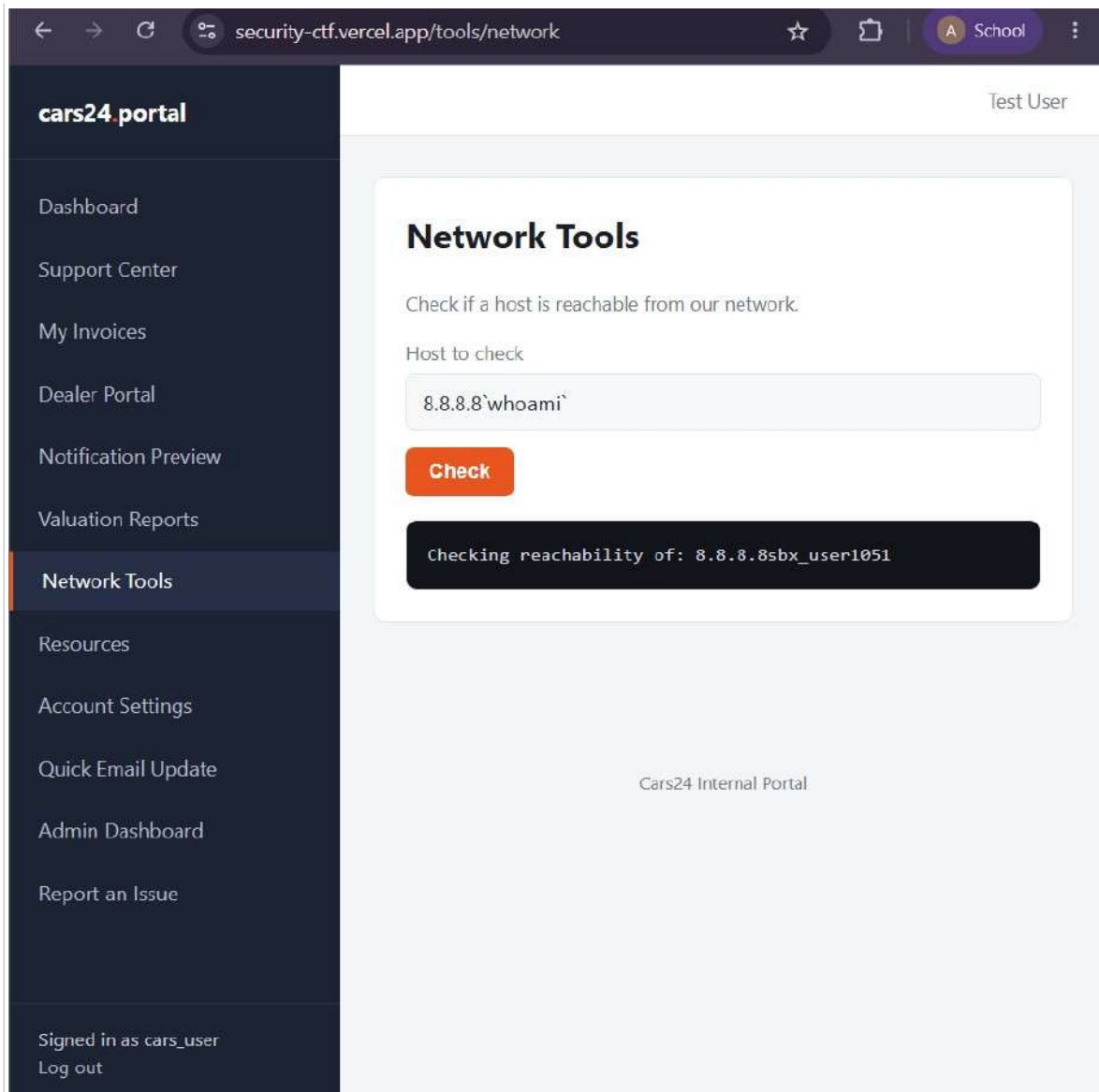


Figure 2.3 — Payload `8.8.8.8`whoami`` returns `sbx_user1051`, corroborating the finding.

Impact

Arbitrary command execution on the application host as user `sbx_user1051` (uid 993). This is full remote code execution: even without root, this level of access routinely enables reconnaissance of the host's filesystem and environment variables (frequently including database credentials or cloud IAM tokens), lateral movement to other internal services reachable from this host, and — depending on host configuration — privilege escalation to root. Testing was deliberately limited to read-only, non-destructive commands (`whoami`, `id`, `hostname`) in line with the engagement's scope; the commands below were not executed, but the access demonstrated is sufficient to run them.

Severity calibration: CVSS base is 8.8 (High). This report rates it Critical because full RCE on the host is the single most severe technical outcome possible and, chained with Finding 1's disclosure, gives an attacker both credentials and code execution — a combination this report treats as critical regardless of the calculator's exact PR/AC inputs.

Remediation

- Never pass user input to a shell. Use a language-native DNS/ICMP library instead of shelling out to ping/nslookup.
- If shelling out is unavoidable, use an allow-list validator for the host format (e.g. strict IP/hostnameregex) and pass arguments via an argv array, never a concatenated string.
- Run this functionality in a tightly sandboxed/least-privilege container with no sensitive data reachable.

FINDING 3 — Insecure Direct Object Reference (IDOR) Exposing PII

CRITICAL

CWE Reference: CWE-639

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N (6.5)

Location: GET /invoices?id=<n>

Description

The My Invoices feature looks up an invoice record purely by the numeric `id` query parameter, with no verification that the invoice belongs to the currently authenticated user. As the logged-in user `cars_user` (own invoice IDs: 1001, 1003), requesting a different, sequential ID returned a complete invoice record belonging to a different customer.

Proof of Concept

Request: GET /invoices?id=1002

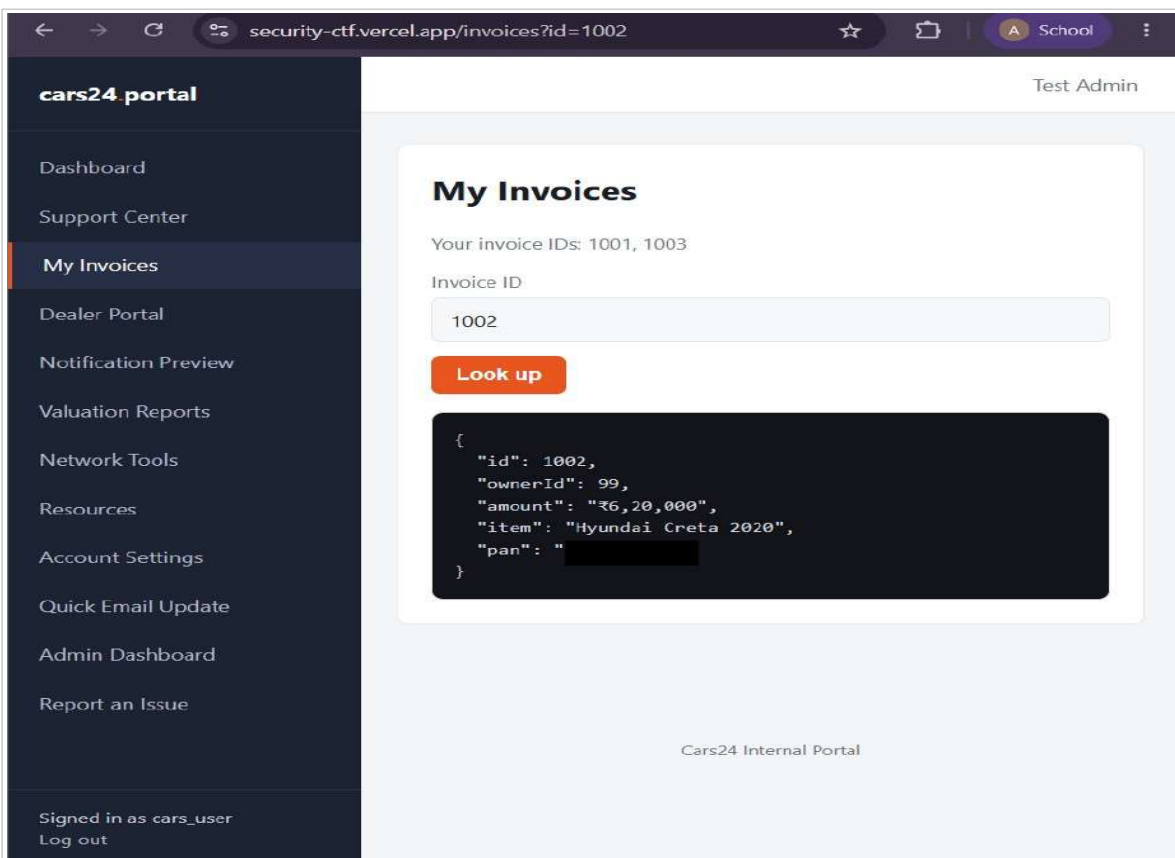


Figure 3.1 — Invoice 1002 (ownerId: 99 — not the logged-in user, ownerId 1) is returned in full, including a PAN (Indian government tax ID, redacted here) and vehicle purchase details. The PAN value is masked in this report; the raw value was captured during testing and is available on request through a secure channel.

Impact

Any authenticated user can enumerate sequential invoice IDs to harvest other customers' Personally Identifiable Information (PII), including PAN numbers, vehicle purchase details, and amounts paid. PAN is government-issued tax ID information in India and is treated as sensitive personal data under the Digital Personal Data Protection (DPDP) Act, 2023; exposure at this scale would likely constitute a reportable data breach in a production system, independent of the purely technical severity.

Severity calibration: CVSS base is 6.5 (Medium). This report rates it Critical because the impact metric does not capture regulatory/privacy exposure — trivially enumerable, government-ID-bearing PII at scale is a business-critical risk even where the CVSS confidentiality-only vector caps the technical score.

Remediation

- Enforce server-side authorization checks on every object lookup: verify the requested invoice's owner ID matches the authenticated session's user ID before returning data.
- Prefer non-sequential, non-guessable identifiers (UUIDs) for user-facing resource references.
- Avoid returning sensitive fields like PAN in API responses unless strictly necessary; mask where possible.

FINDING 4 — Hardcoded Credentials

Disclosed in Page Source

MEDIUM

CWE Reference: CWE-798 / CWE-540

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
(6.5)

Location: GET /login (HTML source comment)

Description

The public, unauthenticated login page ships an HTML comment containing a working username and password for a test account, explicitly labelled as something that should be “removed before this goes to prod”:

```
<!--
Default test account for onboarding (temporary -- remove before this goes to
prod):
username: cars_user
password: cars_password
-->
```

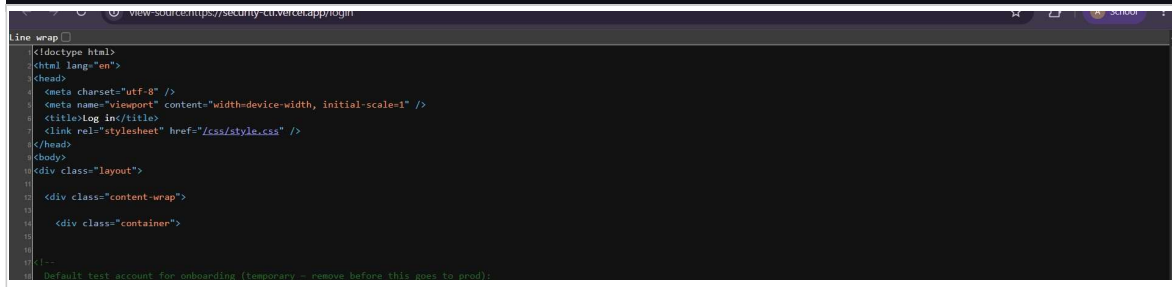


Figure 4.1 — View-source of the public /login page, showing the hardcoded cars_user / cars_password credentials embedded in an HTML comment. This was the entry point for every other finding in this report.

Impact

Anyone who views the page source gains a fully working authenticated session with no need to brute-force or guess credentials. This account was the entry point for every other finding in this report, demonstrating how a seemingly minor disclosure enabled full compromise.

Remediation

- Remove all debug/test comments and credentials from production and staging source before deployment.
- Add a pre-deployment CI check (secret-scanning / grep rule) that fails the build on comments containing "password", "secret", "key", etc.

FINDING 5 — Verbose Database Error / Query Disclosure

MEDIUM

CWE Reference: CWE-209

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
(5.3)

Location: POST /dealer-login

Description

On an unsuccessful login attempt, the Dealer Portal returns the raw backend SQL query — including the submitted username and password values — directly in the HTTP response body.

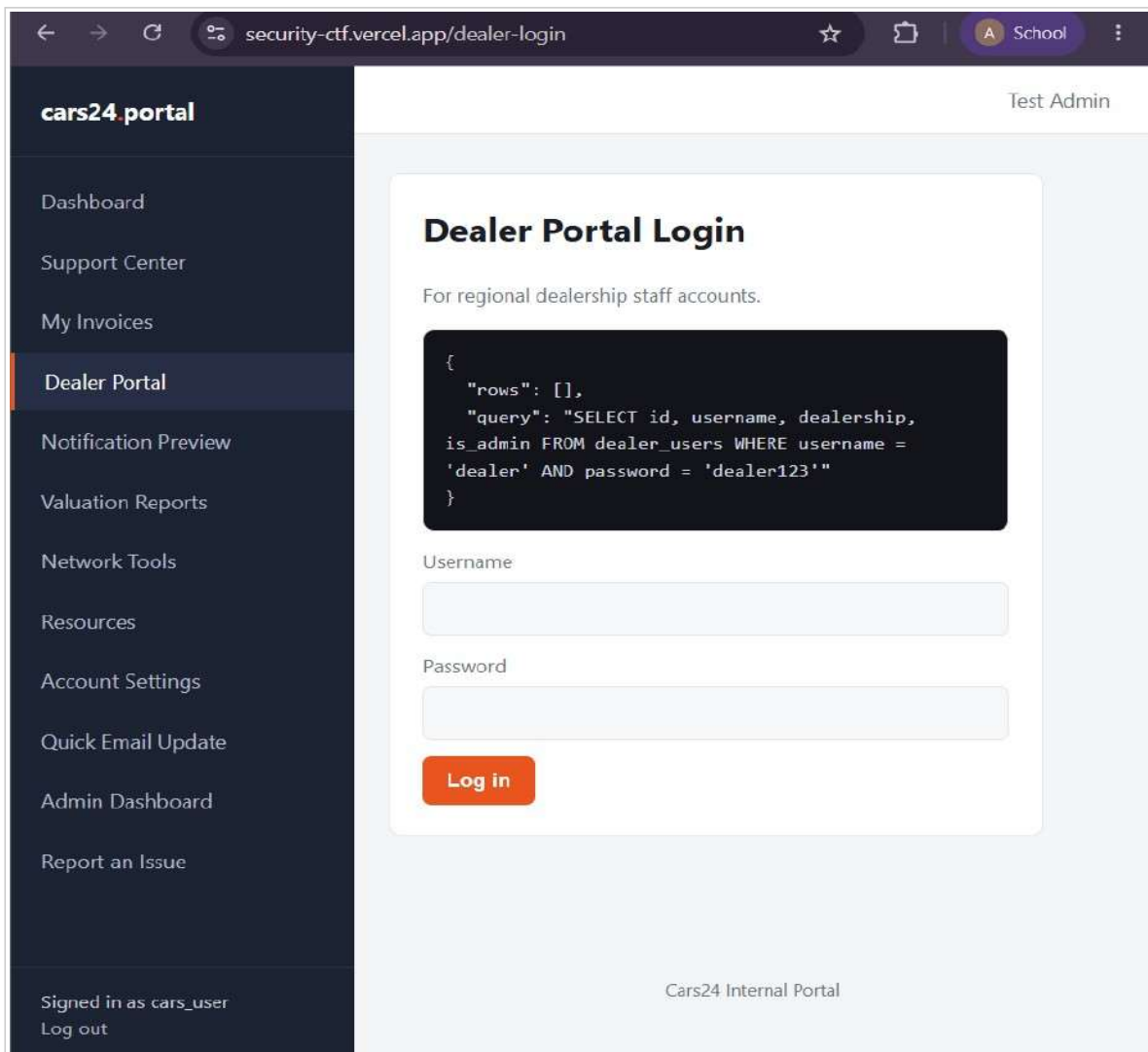


Figure 5.1 — The raw SQL query, including submitted credentials, is reflected back to the client on a failed login (username=dealer, password=dealer123).

Impact

Exposes internal table/column names (`dealer_users`, `is_admin`) and confirms the query is built via string concatenation, directly enabling the SQL injection documented in Finding 1. This is a textbook example of how verbose error disclosure accelerates exploitation of other vulnerabilities.

Remediation

Never return raw database queries, stack traces, or internal error detail to the client. Log detailed errors server-side only and return a generic failure message to the user.

FINDING 6 — Cleartext Password Input Field

LOW

CWE Reference: CWE-549 (related)

CVSS:3.1/AV:P/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N
(1.8)

Location: `/dealer-login` (HTML form)

Description

The Dealer Portal login form defines its password field as a plain text input rather than a masked password input:

```
<input type="text" name="password" />
```

By contrast, the main `/login` form correctly uses `type="password"`.

Impact

The password is rendered in plaintext as the user types, increasing shoulder-surfing risk and the likelihood of accidental exposure via screen-sharing, screenshots, or browser autofill/cache.

Remediation

Change the input type to `password` for all credential fields.

FINDING 7 — Admin Dashboard Correctly Enforces Access Control

INFO

CWE Reference: N/A — Positive Finding

CVSS: n/a — not a vulnerability

Location: `GET /admin`

Despite the `/admin` link being visible in the navigation sidebar for the low-privileged `cars_user` account, directly navigating to it correctly returned `403 - Not authorized`. This control is working as intended and is noted here as a positive finding, not a vulnerability. **Recommendation:** for better UX/security hygiene, consider hiding the link entirely for users without admin rights rather than relying solely on server-side enforcement (defense-in-depth).

FINDING 8 — Server-Side Template Injection — Tested, Not Exploitable

INFO

CWE Reference: CWE-1336 (tested / not present)

CVSS: n/a — not exploitable

Location: POST /tools/notification-preview

The Notification Preview feature accepts a user-defined template containing `{{placeholders}}` and renders a preview. Given the use of template syntax, this endpoint was tested for Server-Side Template Injection using both `{{7*7}}` and `${7*7}` payloads.

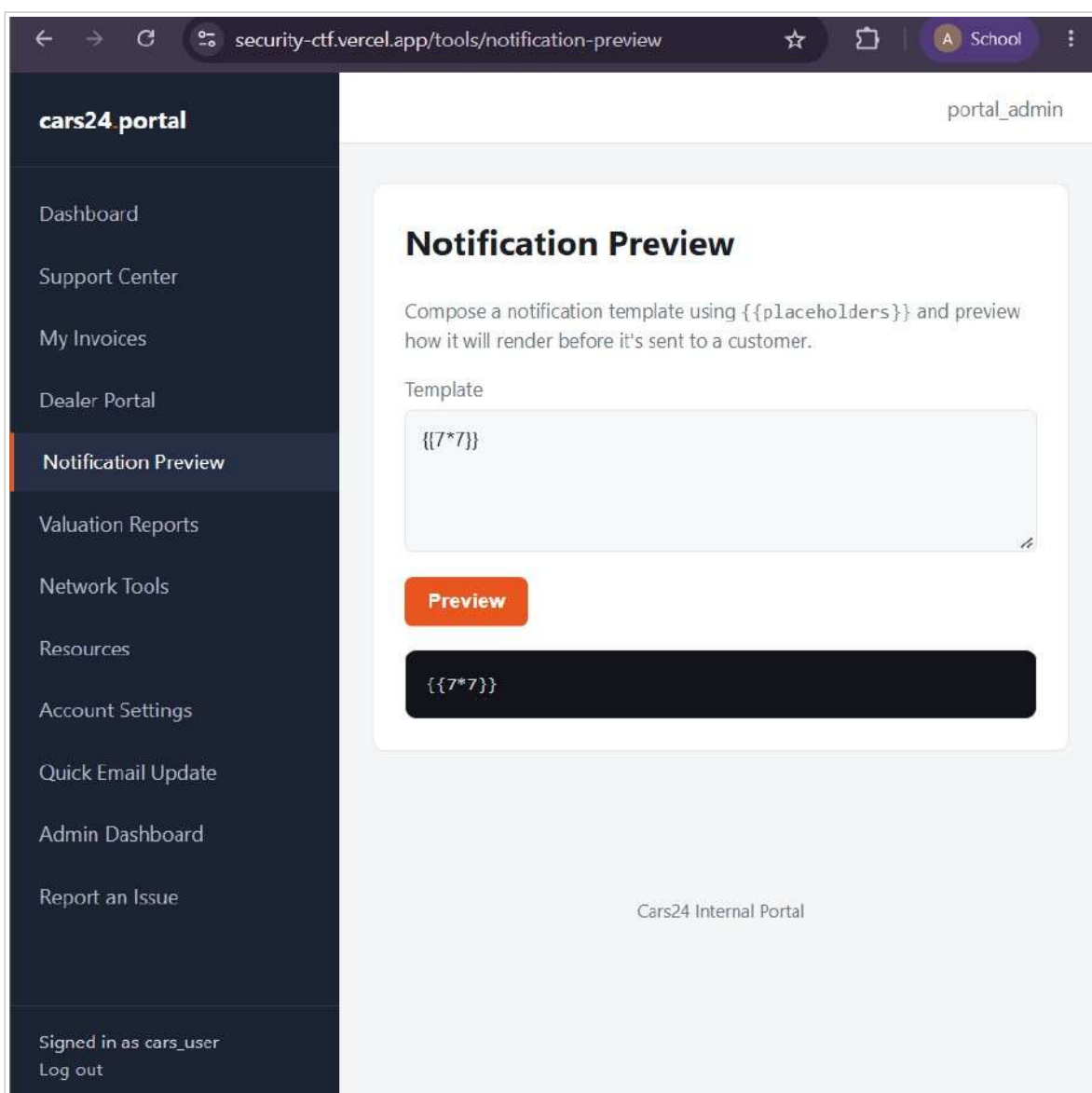


Figure 8.1 — The payload `{{7*7}}` is rendered back literally rather than evaluated to 49, indicating the template engine is not executing injected expressions.

Both payloads were reflected back as literal text rather than evaluated (no 49 appeared in the output), indicating this feature is **not vulnerable** to basic SSTI. This is recorded for completeness and as evidence of testing coverage; no remediation is required.

Risk Matrix

Findings plotted by estimated exploitation likelihood (horizontal) against business impact (vertical). Likelihood reflects how trivial the exploit is to reproduce; impact reflects the worst-case business/regulatory consequence, not the raw CVSS score alone.

Impact ↓ / Likelihood →	Low	Medium	High
High		F2	F1, F3
Medium			F4
Low	F6		F5

F1/F3 (SQLi auth bypass, PII IDOR) sit in the highest-risk cell: trivial to reproduce and high business impact. F2 (RCE) is rated slightly lower on likelihood only because it required discovering that backtick substitution bypassed the semicolon/` filter — its impact ceiling is the same as F1/F3.

Prioritized Remediation Roadmap

Timeframe	Findings	Action
Immediate (24–48 hrs)	F1, F2, F3	Parameterize all SQL queries; remove shell execution from the network-check endpoint (or strictly allow-list host input via argv, never a shell string); add ownerId authorization checks on every /invoices lookup. These three are an active, trivially-reproducible compromise chain.
This Sprint (1–2 weeks)	F4, F5	Strip the hardcoded test credentials from /login source and add a CI secret-scan gate; replace the raw SQL error on /dealer-login with a generic failure message.
Backlog / Hygiene	F6	Change the dealer-login password field to type="password". Low urgency, low effort.
No action required	F7, F8	F7 is a working control — optionally hide the /admin link client-side for defense-in-depth. F8 found no vulnerability; retain as evidence of test coverage.

Conclusion & Overall Risk

The combination of an exposed test credential, unauthenticated-grade SQL injection on a secondary login surface, OS command injection in an internal tool, and an unauthorized-access IDOR exposing government ID numbers represents a critical-risk posture for this application. A single discovered comment in the page source was sufficient to pivot into full data exposure and host-level code execution.

Findings 1–3 should be treated as an emergency, same-week fix — they form a single exploit chain from public page source to remote code execution and regulated-PII disclosure. Findings 4–6 should be addressed in the same remediation cycle as hygiene items that materially lowered the bar for the critical findings above. See the Prioritized Remediation Roadmap for a timeframe-ordered action list.

Testing Methodology Note

All testing was performed manually against the single authenticated low-privilege account discovered via Finding 4, using browser DevTools and manual HTTP manipulation only — no intercepting proxy, automated scanner, brute-force tooling, or load/stress technique was used, in compliance with the engagement's stated scope. Proof-of-concept payloads for command injection were deliberately limited to non-destructive, read-only recon commands. One PII value captured as evidence (Finding 3) has been redacted in this report; the raw value was handled only in-memory during testing and is available to Cars24 through a secure channel on request.